포트폴리오

프로그래머

목차

Unity Enigne

- 프로젝트 설명 - 03
- 서버 구조 - 04
- 로그인 - 05

Direct X

- 프로젝트 설명 - 09
- 던전 - 10
- 최적화 - 14

Unreal Engine

- 프로젝트 설명 - 16
- 벽타기 - 17
- 겪었던 문제 - 20

프로젝트 설명

https://github.com/BIGSUNGG/MMO_RPG

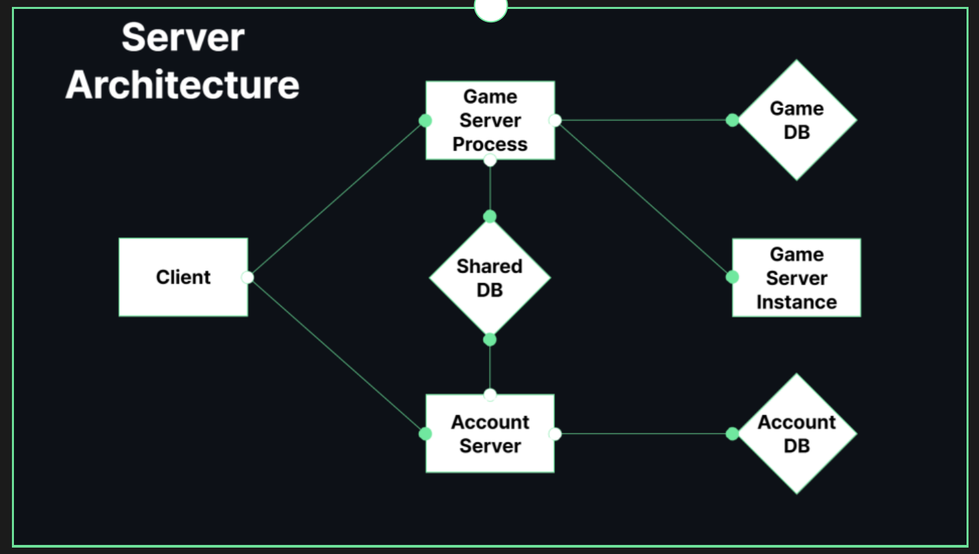
프로젝트 명	MMO RPG
기술 스택	C#, Unity, TCP/IP, HTTP, .NET EF Core, Protobuf
프로젝트 정보	MMO RPG 개인 프로젝트

서버 구조

설명

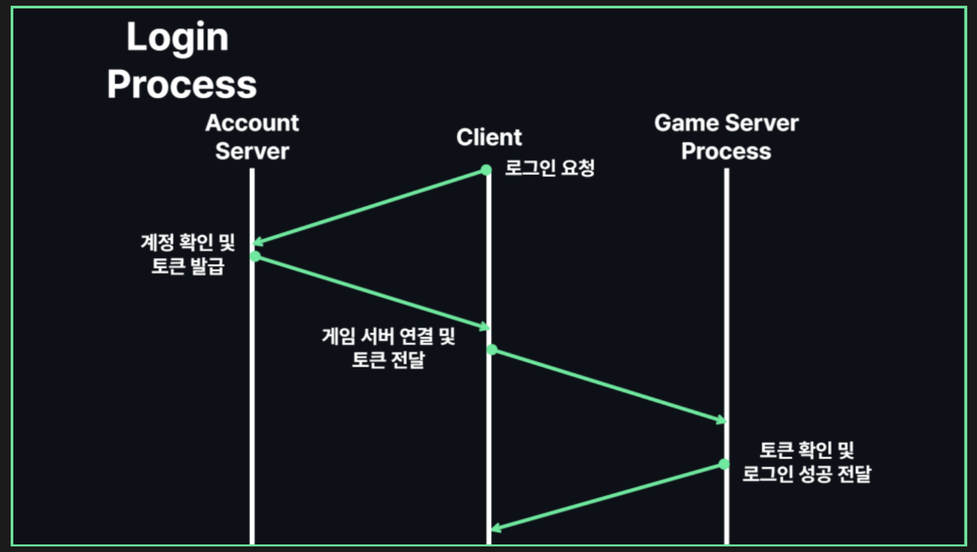
계정 서버는 웹 서버로 HTTP 통신을 하고
로그인, 계정 생성, 토큰 발급을 담당합니다.
게임 서버 프로세스는 TCP/IP 통신을 하고
접속 중인 유저와 게임 서버 인스턴스를 관리합니다.
게임 서버 인스턴스는 Unity Server로
게임 서버 프로세스와의 IPC 통신을 통해
클라이언트와 통신하고 게임의 로직을 담당합니다.

사진



로그인 과정

1. 클라이언트에서 ID와 비밀번호를 담아
계정 서버로 로그인 요청을 하면
2. 계정 서버에서 요청이 올바른지 확인하고
SharedDB에 토큰을 생성한 후
토큰을 클라이언트로 넘깁니다.
3. 계정 서버의 응답을 받은 클라이언트에서
응답에 담겨있는 게임 서버로 연결하고
게임 서버로 응답에 있는 토큰을 전달합니다.
4. 게임 서버에서 SharedDB를 통해
토큰이 올바른지 확인하고 클라이언트에게
로그인 성공 패킷을 전달합니다.



1. 로그인 요청
3-1. 게임 서버 연결

```
public void OnClickLoginButton(PointerEventData evt)
{
    Debug.Log("OnClickLoginButton");

    string account = Get<GameObject>((int)GameObjects.AccountName).GetComponent<InputField>().text;
    string password = Get<GameObject>((int)GameObjects.Password).GetComponent<InputField>().text;


    LoginAccountPacketReq packet = new LoginAccountPacketReq()
    {
        AccountName = account,
        Password = password
    };

    Managers.Web.SendPostRequest<LoginAccountPacketRes>("account/login", packet, (res) =>
    {
        Debug.Log(res.LoginOk);
        Get<GameObject>((int)GameObjects.AccountName).GetComponent<InputField>().text = "";
        Get<GameObject>((int)GameObjects.Password).GetComponent<InputField>().text = "";

        if (res.LoginOk)
        {
            Managers.Network.AccountName = account;
            Managers.Network.AccountId = res.AccountId;
            Managers.Network.Token = res.Token;

            Managers.Network.ConnectToGame(res.ServerList[0]);
            Managers.UI.ClosePopupUI();
        }
    });
}
```

```
public static void S_ConnectedHandler(ISession session, IMessage packet)
{
    S_Connected recvPacket = packet as S_Connected;

    // 로그인 패킷 보내기
    C_Login loginPacket = new C_Login();

#if true // 정상적인 패킷 보내기
    loginPacket.AccountId = Managers.Network.AccountId;
    loginPacket.Token = Managers.Network.Token;
#else // 비정상적인 패킷 보내기
    loginPacket.AccountId = 11231223;
    loginPacket.Token = 129212222;
#endif

    session.Send(loginPacket);
}
```

3-2. 토큰을 담은 로그인 시도 패킷 전달

2. 계정 확인 및 토큰 발급

```
[Route("login")]
public LoginAccountPacketRes LoginAccount([FromBody] LoginAccountPacketReq req)
{
    LoginAccountPacketRes res = new LoginAccountPacketRes();

    AccountDb account = _context.Accounts
        .AsNoTracking()
        .Where(a => a.LoginId == req.AccountName && a.Password == req.Password)
        .FirstOrDefault();

    if (account == null)
    {
        Console.WriteLine($"Login Fail Id : {req.AccountName}, Password : {req.Password}");
        res.LoginOk = false;
    }
    else
    {
        Console.WriteLine($"Login Success Id : {req.AccountName}, Password : {req.Password}");
        res.LoginOk = true;

        // 토큰 발급
        DateTime expired = DateTime.UtcNow;
        expired.AddSeconds(600);

        TokenDb tokenDb = _shared.Tokens.Where(t => t.AccountDbId == account.AccountDbId).FirstOrDefault();
        if (tokenDb != null)
        {
            tokenDb.Token = new Random().Next(Int32.MinValue, Int32.MaxValue);
            tokenDb.Expired = expired;
            _shared.SaveChangesEx();
        }
        else
        {
            tokenDb = new TokenDb()
            {
                AccountDbId = account.AccountDbId,
                Token = new Random().Next(Int32.MinValue, Int32.MaxValue),
                Expired = expired
            };
            _shared.Add(tokenDb);
            _shared.SaveChangesEx();
        }

        res.AccountId = account.AccountDbId;
        res.Token = tokenDb.Token;
        res.ServerList = new List<ServerInfo>();

        foreach (ServerDb serverDb in _shared.Servers)
        {
            res.ServerList.Add(new ServerInfo()
            {
                Name = serverDb.Name,
                IpAddress = serverDb.IpAddress,
                Port = serverDb.Port,
                BusyScore = serverDb.BusyScore
            });
        }
    }

    return res;
}
```

4-1. 로그인 시도 패킷 처리

```
public static void C_LoginHandler(ISession session, IMessage packet)
{
    C_Login recvPacket = packet as C_Login;

    ClientSession clientSession = session as ClientSession;
    if (clientSession == null)
        return;

    clientSession.LoginAccount(recvPacket);
}
```

```
public void LoginAccount(C_Login loginPacket)
{
    if (LoginState != PlayerLoginState.NotLoggedIn)
        return;

    Console.WriteLine("Login Try");

    // 패킷에 맞는 토큰 찾기
    TokenDb findToken = null;
    using (SharedDbContext db = new SharedDbContext())
    {
        findToken = db.Tokens
            .Where(t => t.Token == loginPacket.Token && t.AccountDbId == loginPacket.AccountId).FirstOrDefault();

        if (findToken == null) // 토큰을 못찾았을 경우
        {
            Console.WriteLine("Failed to find token");
            LoginFail();
            return;
        }
    }

    // 토큰에 맞는 계정 찾기
    GameAccountDb findAccount = null;
    using (GameDbContext db = new GameDbContext())
    {
        findAccount = db.Accounts
            .Where(a => a.GameAccountDbId == findToken.AccountDbId).FirstOrDefault();
    }

    if (findAccount == null) // 계정을 못찾았을 경우
    {
        Console.WriteLine("Failed to find account");

        // 계정 만들기
        findAccount = CreateAccount(findToken.AccountDbId);
        if (findAccount == null) // 계정을 생성이 실패했을 경우
        {
            LoginFail();
            return;
        }
    }
    else // 계정이 있을 경우
    {
        ClientSession session = GameAccountManager.Instance.Find(findAccount.GameAccountDbId);
        if (session != null && session.LoginState == PlayerLoginState.LoggedIn) // 계정이 다른 클라이언트에서 이미 플레이 중이라면
        {
            Console.WriteLine("Already playing this account");

            // 이미 연결중인 클라이언트 연결해제
            session.Disconnect();
        }
    }

    LoginSuccess(findAccount.GameAccountDbId);
}
```

4-2. 토큰 확인 및 같은 계정으로 연결 중인 클라이언트 연결 해제

4-3. 로그인 성공 패킷 전송

```
169         private void LoginSuccess(int gameAccountDbId)
170         {
171             Console.WriteLine("Login Success");
172             GameAccountDbId = gameAccountDbId;
173             LoginState = PlayerLoginState.LoggedIn;
174
175             // 로그인 성공 패킷 보내기
176             S_Login sendPacket = new S_Login();
177             sendPacket.LoginResult = LoginResult.LoginSuccess;
178             Send(sendPacket);
179
180             GameAccountDb loginAccount = null;
181             using (GameDbContext db = new GameDbContext())
182             {
183                 loginAccount = db.Accounts
184                     .AsNoTracking()
185                     .Include(a => a.Player)
186                     .Where(a => a.GameAccountDbId == gameAccountDbId).FirstOrDefault();
187             }
188
189             if (loginAccount == null)
190             {
191                 Console.WriteLine("Login account is null");
192                 return;
193             }
194
195             if (loginAccount.Player == null)
196             {
197                 Console.WriteLine("Login account's player is null");
198                 loginAccount.Player = CreatePlayer(gameAccountDbId);
199
200                 if(loginAccount.Player == null)
201                     return;
202             }
203
204             StartSession();
205         }
```

4-4. 유저 정보 불러오기

```
247         private void StartSession()
248         {
249             // 계정 정보가 업데이트 됐을경우
250             if (GameAccountManager.Instance.IsSaved(this.GameAccountDbId))
251             {
252                 // 계정 정보에 맞게 세션 정보 업데이트
253                 GameAccountDb loginAccount = null;
254                 using (GameDbContext db = new GameDbContext())
255                 {
256                     loginAccount = db.Accounts
257                         .AsNoTracking()
258                         .Include(a => a.Player)
259                         .ThenInclude(p => p.ItemSlot)
260                         .Where(a => a.GameAccountDbId == this.GameAccountDbId).FirstOrDefault();
261                 }
262
263                 if (loginAccount == null)
264                 {
265                     Console.WriteLine("LoginAccount is null");
266                     return;
267                 }
268
269                 if (loginAccount.Player == null)
270                 {
271                     Console.WriteLine("LoginAccount's Player is null");
272                     return;
273                 }
274
275                 // 플레이어 정보 업데이트
276                 SetPlayerInfo(loginAccount.Player);
277
278                 // 접속중인 계정 추가하기
279                 GameAccountManager.Instance.Add(this.GameAccountDbId, this);
280
281                 // 핑 패킷 보내기
282                 GameInstanceManager.Instance.PushAfter(5000, Ping);
283
284                 // 유저를 GameMap에 추가
285                 GameInstanceManager.Instance.Push(() => { EnterMap(GameInstanceManager.Instance.Find(loginAccount.Player.MapId), 0); });
286
287                 Console.WriteLine("Success to start session");
288             }
289             else
290             {
291                 GameInstanceManager.Instance.PushAfter(250, StartSession);
292             }
293         }
```

프로젝트 설명

https://github.com/BIGSUNGG/DirectX_2D

프로젝트 명	Copy Dungreed
기술 스택	C++, DirectX 11, FMOD, ImGui
프로젝트 정보	던그리드 모작 개인 프로젝트.

던전

설명

본 게임 모드 시작 시
랜덤한 던전을 생성하여
시작 맵에서 다양한 맵으로
이동할 수 있습니다.

사진

던전 생성 코드 (1)

던전 정보를 초기화하고
현재 레벨의 모든 맵을 불러오고
던전에 기본 맵을 추가합니다.

```
void MapManager::MakeRandomMap(int level, int num)
{
    // 미로 초기화
    _maps.clear();
    _mapCount = 0;
    _curMapPos = { 0,0 };

    for (int x = -_mapHalfSize.x; x <= _mapHalfSize.x; x++)
    {
        for (int y = -_mapHalfSize.y; y <= _mapHalfSize.y; y++)
        {
            _maps[x][y] = nullptr;
        }
    }

    vector<MapBasic> curLevelMapList;
    curLevelMapList.reserve(_mapList[level].size() - 1);
    for (int i = 1; i < _mapList[level].size(); i++)
    {
        int num = _mapList[level][i];
        MapBasic info = LoadBasicInfo(level, num);
        curLevelMapList.push_back(info);
    }

    // 기본 맵 추가
    shared_ptr<Map> curMap = LoadMap(level, num);
    const Vector2 startPos = { 0, 0 };
    Vector2 curPos = startPos;
    AddMap(curMap, curPos);
    queue<Vector2> visitQueue;
```

던전 생성 코드 (2)

가장 최근에 추가된 맵에서
갈 수 있는 장소를 BFS 알고리즘을
이용하여 순회하다 갈 수 있는 위치에
맵이 할당되어있지 않다면
조건에 맞는 맵을 구하고 추가합니다.

```
while (true)
{
    // 최근 추가된 맵에서 갈 수 있는 위치 추가
    if (curMap)
    {
        if (curMap->CanGoTop())
            visitQueue.push(curPos + Vector2(0, 1));
        if (curMap->CanGoBottom())
            visitQueue.push(curPos + Vector2(0, -1));
        if (curMap->CanGoLeft())
            visitQueue.push(curPos + Vector2(-1, 0));
        if (curMap->CanGoRight())
            visitQueue.push(curPos + Vector2(1, 0));

        curMap = nullptr;
    }

    if (visitQueue.size() <= 0) // 더 이상 맵을 놓을 수 없다면
        break;

    curPos = visitQueue.front();
    visitQueue.pop();
    if (_maps[curPos.x][curPos.y] != nullptr) // 현재 있는 위치에 맵이 이미 있다면
        continue;

    // 해당 방향으로 무조건 연결되어야하는지 확인
    bool shouldCanGoTop = _maps[curPos.x][curPos.y + 1] ? _maps[curPos.x][curPos.y + 1]->CanGoBottom() : false;
    bool shouldCanGoBottom = _maps[curPos.x][curPos.y - 1] ? _maps[curPos.x][curPos.y - 1]->CanGoTop() : false;
    bool shouldCanGoLeft = _maps[curPos.x - 1][curPos.y] ? _maps[curPos.x - 1][curPos.y]->CanGoRight() : false;
    bool shouldCanGoRight = _maps[curPos.x + 1][curPos.y] ? _maps[curPos.x + 1][curPos.y]->CanGoLeft() : false;

    // 적절한 맵 구하기
    MapBasic* info = nullptr;
    while (true)
    {
        // 랜덤한 맵 구하기
        info = &curLevelMapList[MathUtility::RandomInt(0, curLevelMapList.size() - 1)];

        // 맵이 필요한 조건에 맞는지
        if (shouldCanGoTop && info->CanGoTop() == false)
            continue;
        if (shouldCanGoBottom && info->CanGoBottom() == false)
            continue;
        if (shouldCanGoLeft && info->CanGoLeft() == false)
            continue;
        if (shouldCanGoRight && info->CanGoRight() == false)
            continue;

        // 맵에서 이동할 수 있는 위치가 맵의 크기를 넘는다면
        if (info->CanGoTop() && curPos.y >= _mapHalfSize.y)
            continue;
        if (info->CanGoBottom() && curPos.y <= -_mapHalfSize.y)
            continue;
        if (info->CanGoRight() && curPos.x >= _mapHalfSize.x)
            continue;
        if (info->CanGoLeft() && curPos.x <= -_mapHalfSize.x)
            continue;

        // 맵에서 이동할 수 있는 맵이 있고 그 맵과 이어지지않다면
        shared_ptr<Map> topMap    = _maps[curPos.x][curPos.y + 1];
        shared_ptr<Map> bottomMap = _maps[curPos.x][curPos.y - 1];
        shared_ptr<Map> rightMap  = _maps[curPos.x + 1][curPos.y];
        shared_ptr<Map> leftMap   = _maps[curPos.x - 1][curPos.y];

        if(info->CanGoTop() && topMap && topMap->CanGoBottom() == false)
            continue;
        if (info->CanGoBottom() && bottomMap && bottomMap->CanGoTop() == false)
            continue;
        if (info->CanGoRight() && rightMap && rightMap->CanGoLeft() == false)
            continue;
        if (info->CanGoLeft() && leftMap && leftMap->CanGoRight() == false)
            continue;

        curMap = LoadMap(info->_level, info->_num);
        AddMap(curMap, curPos);
        break;
    }
```

던전 생성 코드 (3)

맵을 충분히 추가했다면
시작 맵으로부터 BFS알고리즘을
이용하여 순회하다 갈 수 있는 위치에
맵이 할당되어있지 않다면
막다른길이 될 수 있는
맵을 구하고 추가합니다.

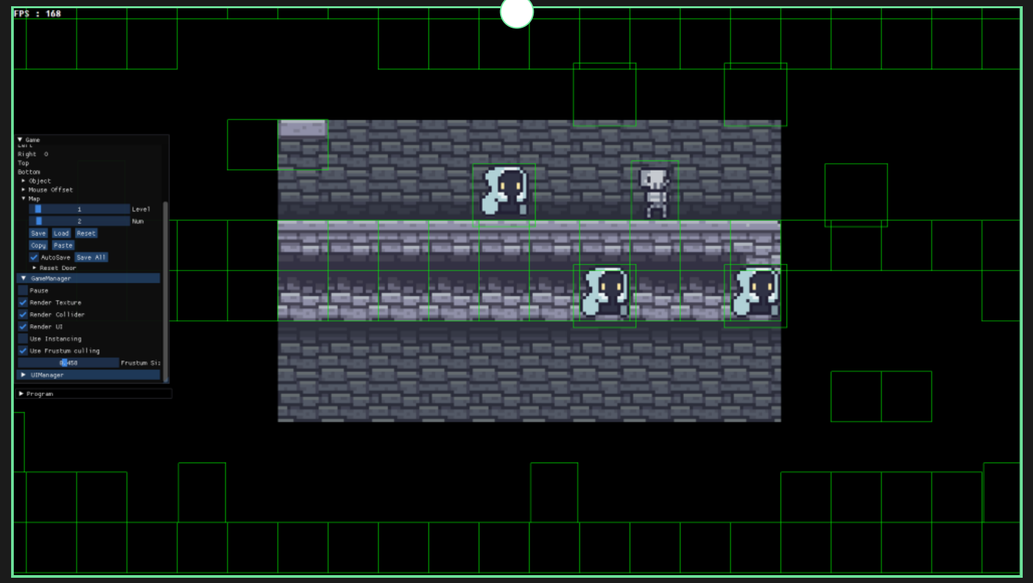
```
        // 맵을 충분히 추가했다면
        if (_mapCount >= _mapCountMin)
        {
            // 공백에 연결되어있는 맵 찾기
            queue<Vector2> visitQueue;
            unordered_map<int, unordered_map<int, bool>> visited;
            Vector2 curPos = startPos;
            shared_ptr<Map> curMap = _maps[startPos.x][startPos.y];

            while (true)
            {
                // 최근 추가된 맵에서 갈 수 있는 위치 추가
                if (curMap)
                {
                    if (curMap->CanGoTop())
                        visitQueue.push(curPos + Vector2(0, 1));
                    if (curMap->CanGoBottom())
                        visitQueue.push(curPos + Vector2(0, -1));
                    if (curMap->CanGoLeft())
                        visitQueue.push(curPos + Vector2(-1, 0));
                    if (curMap->CanGoRight())
                        visitQueue.push(curPos + Vector2(1, 0));

                    curMap = nullptr;
                }

                if (visitQueue.size() <= 0) // 더 이상 확인할 맵이 없다면
                    break;

                curPos = visitQueue.front();
                visitQueue.pop();

                if (visited[curPos.x][curPos.y] == true) // 이미 확인한 위치라면
                    continue;

                visited[curPos.x][curPos.y] = true;

                curMap = _maps[curPos.x][curPos.y];
                if (curMap == nullptr) // 현재 있는 위치에 맵이 없다면
                {
                    // 연결되야하는 방향 구하기
                    bool shouldCanGoTop = _maps[curPos.x][curPos.y + 1] ? _maps[curPos.x][curPos.y + 1]->CanGoBottom() : false;
                    bool shouldCanGoBottom = _maps[curPos.x][curPos.y - 1] ? _maps[curPos.x][curPos.y - 1]->CanGoTop() : false;
                    bool shouldCanGoLeft = _maps[curPos.x - 1][curPos.y] ? _maps[curPos.x - 1][curPos.y]->CanGoRight() : false;
                    bool shouldCanGoRight = _maps[curPos.x + 1][curPos.y] ? _maps[curPos.x + 1][curPos.y]->CanGoLeft() : false;

                    // 적절한 맵 구하기
                    MapBasic* info = nullptr;
                    while (true)
                    {
                        // 랜덤한 맵 구하기
                        info = &curLevelMapList[MathUtility::RandomInt(0, curLevelMapList.size() - 1)];

                        // 맵이 필요한 조건에 맞는지
                        if (shouldCanGoTop != info->CanGoTop())
                            continue;
                        if (shouldCanGoBottom != info->CanGoBottom())
                            continue;
                        if (shouldCanGoLeft != info->CanGoLeft())
                            continue;
                        if (shouldCanGoRight != info->CanGoRight())
                            continue;

                        curMap = LoadMap(info->_level, info->_num);
                        AddMap(curMap, curPos);
                        break;
                    }
                }
            }
            break;
        }
    }

    SetCurMap(startPos);
}
```

프러스텀 컬링

설명

카메라 외부에 있는 오브젝트를
AABB 충돌로 확인하여 렌더링에서 제외하는
프러스텀 컬링을 적용하여 일반적인 상황에서
평균 FPS가 60에서 110으로,
또한 모든 오브젝트들이 제외되는 상황에서는
최대 340으로 향상하였습니다.

사진



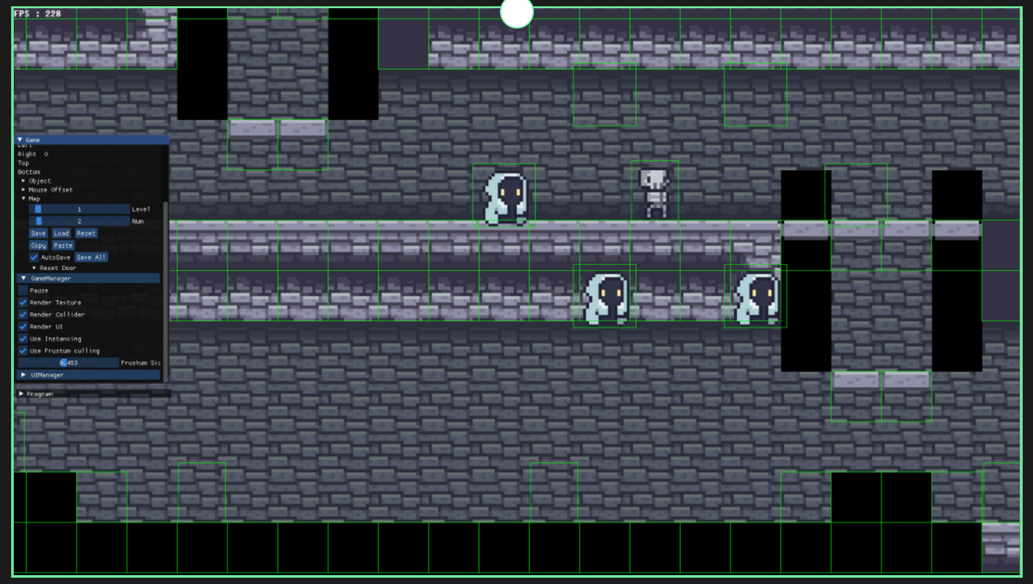
인스턴싱 X, 프러스텀 컬링 O, AABB 크기 0.45배

인스턴싱

설명

동일한 텍스쳐를 가진 오브젝트에
한번의 드로우콜로 렌더링할 수 있는
인스턴싱을 사용하여
프러스텀 컬링을 비활성화 해놓은 환경을
기준으로 평균 FPS가
60에서 360으로 향상되었습니다

사진



인스턴싱 O, 프러스텀 컬링 O, AABB 크기 0.45배

프로젝트 설명

https://github.com/BIGSUNGG/FPS

프로젝트 명	Mutlplayer FPS
기술 스택	C++, Unreal Engine, OnlineSubsystem(UE)
프로젝트 정보	멀티플레이 FPS 개인 프로젝트

벽타기

설명

점프 버튼을 누른 상태로
벽에 붙어 벽의 방향으로
벽타기를 시작합니다.

사진

벽타기 코드 (1)

WallRunHorizonUpdate에서
좌, 우 벽타기를 시도하며
벽타기 성공 시 Server RPC 함수인
Server_WallRunHorizonUpdate 함수를 호출하여
서버와 클라이언트의 싱크를 맞춥니다.

```
bool UAdvanceMovementComponent::WallRunUpdate()
{
    if(CurWallRunState != EWallRunState::WALLRUN_VERTICAL) // 세로 벽타기 중이 아니라면
        CurWallRunVerticalSpeed = FMath::FInterpTo(CurWallRunVerticalSpeed, MaxWallRunVerticalSpeed, GetWorld()->GetDeltaSeconds(), 5.f); // 세로 벽타기 속도 복구

    if (OwnerCreature->GetbJumpButtonPress() || IsWallRunning()) // 점프 버튼을 누르고있거나 벽타기 중이면
    {
        bool WallRunVerticalSuccess = false;
        bool WallRunHorizonSuccess = false;

        if (!bWallRunVerticalSupressed) // 세로 벽타기가 가능한지
        {
            WallRunVerticalSuccess = WallRunVerticalUpdate(); // 세로 벽타기 시작
            if (WallRunVerticalSuccess) // 벽타기 성공했을때
            {
                FVector WallRunDir = PendingLaunchVelocity;
                WallRunDir.Normalize();
                Server_WallRunVerticalUpdate(WallRunDir, CurWallRunVerticalSpeed);
                return true;
            }
        }

        if (!bWallRunHorizonSupressed) // 가로 벽타기가 가능한지
        {
            WallRunHorizonSuccess = WallRunHorizonUpdate(); // 가로 벽타기 시작
            if (WallRunHorizonSuccess) // 벽타기 성공했을때
            {
                FVector WallRunDir = PendingLaunchVelocity;
                WallRunDir.Normalize();
                Server_WallRunHorizonUpdate(WallRunDir);
                return true;
            }
        }
    }

    return false;
}
```

```
bool UAdvanceMovementComponent::WallRunHorizonUpdate()
{
    if ((CurWallRunState == EWallRunState::NONE || CurWallRunState == EWallRunState::WALLRUN_RIGHT) &&
        WallRunHorizonMovement(OwnerCreature->GetActorLocation(), CalculateRightWallRunEndVector(), -1.f))
        // 오른쪽 벽타기 확인
    {
        if (!IsWallRunning())
            WallRunStart(EWallRunState::WALLRUN_RIGHT);

        return true;
    }
    else if (CurWallRunState == EWallRunState::WALLRUN_RIGHT)
        // 오른쪽 벽타기 중이였지만 오른쪽 벽타기를 실패한경우
    {
        WallRunHorizonEnd(1.f);
        return false;
    }
    else if ((CurWallRunState == EWallRunState::NONE || CurWallRunState == EWallRunState::WALLRUN_LEFT) &&
        WallRunHorizonMovement(OwnerCreature->GetActorLocation(), CalculateLeftWallRunEndVector(), 1.f))
        // 왼쪽 벽타기 확인
    {
        if (!IsWallRunning())
            WallRunStart(EWallRunState::WALLRUN_LEFT);

        return true;
    }
    else if (CurWallRunState != EWallRunState::WALLRUN_LEFT)
        // 왼쪽 벽타기 중이였지만 왼쪽 벽타기를 실패한경우
    {
        WallRunHorizonEnd(1.f);
        return false;
    }

    return false;
}
```

벽타기 코드 (2)

WallRunHorizonMovement 함수에서
벽타기를 할 벽을 찾고
벽의 방향과 Up벡터와 외적을 하여 나온 법선벡터에
오른쪽 벽타기라면 -1을 곱해주어
이동할 방향으로 정합니다.
이후 이동할 방향에 벽타기 속도를 곱해주어
최종적으로 적용할 속도로 적용합니다.

이후에 Server_WallRunHorizonUpdate가
호출되고 Validate함수에서 벽타기 중인지 확인하고
Implementation 함수에서 속도를 설정합니다.

```
bool UAdvanceMovementComponent::WallRunHorizonMovement(FVector Start, FVector End, float WallRunDirection)
{
    if (bWallRunHorizonSupressed)
        return false;

    // 벽타기할 오브젝트 찾기
    FCollisionQueryParams ObjectParams(NAME_None, false, OwnerCreature);
    FHitResult Result;
    bool bSuccess = GetWorld()->LineTraceSingleByChannel(
        Result,
        Start,
        End,
        ECC_ONLY_OBJECT,
        ObjectParams
    );

    if (bSuccess && Result.bBlockingHit)
    {
        if ((IsFalling() || IsWallRunning()) && CalculateValidWallVector(Result.Normal))
        {
            // 벽 각도에 맞춰 이동
            WallRunNormal = Result.Normal;
            FVector WallRunVec = FVector::CrossProduct(WallRunNormal, FVector(0, 0, 1)) * (WallRunDirection * WallRunHorizonSpeed);
            OwnerCreature->LaunchCharacter(WallRunVec, true, !bWallRunGravity);

            return true;
        }
    }
    return false;
}
```

```
void UAdvanceMovementComponent::Server_WallRunHorizonUpdate_Implementation(FVector Direction)
{
    Direction.Normalize();
    PendingLaunchVelocity = Direction * WallRunHorizonSpeed;
}

bool UAdvanceMovementComponent::Server_WallRunHorizonUpdate_Validate(FVector Direction)
{
    if (CurWallRunState == EWallRunState::WALLRUN_LEFT)
        return true;

    if (CurWallRunState == EWallRunState::WALLRUN_RIGHT)
        return true;

    return false;
}
```

겪었던 문제

문제 & 이유

무기의 장착, 손에 들기, 집어넣기와 같은 정보를
NetMutlicast 함수를 사용하여
클라이언트에게 알리는 방법을 사용하니
늦게 게임에 접속하는 등의 이유로
Rpc 함수를 전달 받지 못한 클라이언트에서
변경된 정보를 반영하지 못하는
문제가 발생하였습니다.

해결 방법

ReplicatedUsing 기능을 사용하여
무기 슬롯에 아이템이 추가되면 장착
현재 무기가 변경되면 이전의 무기를 집어넣고
현재 무기를 손에 드는 등의 방법으로 리팩토링하여
클라이언트에서 변경된 정보를 반영하지 못하는
문제를 해결하였습니다.